

1 Áttekintés

A feladat egy N csúcsból és M élből álló gráfot vizsgál, ahol minden csúcsnak és élnek van egy súlya. A probléma lényege, hogy minden csúcsra ki kell számítani egy értéket, amely valamilyen módon függ a rajta keresztül haladó legmagasabb súlyú hidak útjától.

2 kulcsfontosságú megfigyelések

2.1 Hidak keresése

Az algoritmus elsőként meghatározza a gráf összes hídját. Egy híd az az él, amelynek eltávolítása szétválasztja a gráfot. Ezt a Tarjan-féle mélységi kereséssel oldjuk meg: minden csúcsra tároljuk a belépési szintjét (lvl) és az alatta elérhető minimális szintet ($lvlMin$). Ha egy gyermek csúcs $lvlMin$ értéke megegyezik a szülő szintjével, akkor a köztük lévő él híd.

2.2 Komponensekre bontás

A hidak eltávolítása után a gráf 2-kapcsolt komponensekre esik szét. Minden komponensen belül bármely két csúcs között van legalább két út, így ezek a komponensek nem érzékenyek egyetlen él elvesztésére sem.

2.3 A hidak fa-struktúrája

A 2-kapcsolt komponensek közötti hidak fa-struktúrát alkotnak. A hidak súly szerinti csökkenő sorrendben való feldolgozása kulcsfontosságú: amikor egy híd feldolgozunk, a hozzá kapcsolódó komponensek még nem voltak összekapcsolva, így garantáltan helyes eredményt kapunk.

3 A megoldás lépései

1. **Hídazonosítás:** DFS-sel megkeressük az összes hídát, és tároljuk azokat a *bridges* vektorban.
2. **Komponensépítés:** A hidak eltávolításával meghatározzuk a 2-kapcsolt komponenseket. Minden csúcsához hozzárendeljük a komponensét (*comp*).
3. **Hidak feldolgozása csökkenő sorrendben:** A hidakat súly szerint rendezzük (legnagyobb súly először). Kezdetben minden komponens saját halmaz. Unió-Find struktúrával összekapcsoljuk a komponenseket.
4. **Értékek propagálása:** A kapott fa-struktúrában propagáljuk az akkumulált értékeket a gyökértől a levelekig.

4 Adatszerkezetek részletesen

Az Union-Find implementáció speciális tulajdonságokkal bír a feladathoz:

- $cnt[root]$: a halmaz mérete
- $val[root]$: az akkumulált érték a halmazban
- Összeuniózáskor a kisebb halmazt a nagyobbhoz kapcsoljuk (heurisztika)
- Az összeuniózás során: $val[y] + = 1LL \cdot cst \cdot cnt[x]$ és $val[x] + = 1LL \cdot cst \cdot cnt[y] - val[y]$

5 Korrektség indoklása

5.1 Híd tulajdonság kihasználása

A feladat lényege, hogy minden csúcshoz a hozzá vezető legmagasabb súlyú híd útját kell figyelembe venni. A 2-kapcsolt komponenseken belül nincs híd, ezért egy komponens bármely csúcsa azonos útvonalakat lát a hidakon keresztül.

5.2 Sú szerinti feldolgozás

A hidakat csökkenő sorrendben dolgozzuk fel. Amikor egy hidat feldolgozunk:

- A két oldalon lévő komponensek még nem kapcsolódtak össze korábban
- Az összekapcsolás pillanatában az aktuális súly a maximális híd-súly az útvonalon
- A val tömbben tárolt értékek pontosan az ennél kisebb súlyú hidak hatását tükrözik

5.3 Propagálás helyessége

A hidak feldolgozása során kapott fa-struktúrában minden értéket a gyökértől propagálunk. Mivel a fa pontosan a híd-kapcsolatokat reprezentálja, és minden él pontosan egyszer kerül feldolgozásra, minden csúcs megkapja az összes szükséges értéket.

6 Idő- és térkomplexitás

6.1 Időkomplexitás

- Hídkeresés: $O(N + M)$
- Komponensek meghatározása: $O(N + M)$
- Rendezés: $O((N + M) \log(N + M))$

- Union-Find műveletek: $O((N + M) \log^*(N + M))$

Összesen: $O((N + M) \cdot \log(N + M))$

6.2 Térkomplexitás

- Gráf tárolása: $O(N + M)$
- További tömbök: $O(N)$

Összesen: $O(N + M)$

7 Megvalósítási megjegyzések

A kódban fontos megfigyelések:

```
// H d keres se
void getBridges(int nod, int parent) {
    for (edge& vec : g[nod])
        if (lvl[vec.x] == 0) {
            lvl[vec.x] = lvlMin[vec.x] = lvl[nod] + 1;
            getBridges(vec.x, nod);
            if (lvlMin[vec.x] == lvl[vec.x]) {
                isBridge[vec.pos] = true;
                bridges.push_back({nod, vec.x, vec.c});
            }
            lvlMin[nod] = min(lvlMin[nod], lvlMin[vec.x]);
        }
    else if (vec.x != parent) lvlMin[nod] = min(lvlMin[nod], lvl[vec.x]);
}
```

A csúcsok súlyát speciális "-1" x koordinátával tároljuk a *bridges* vektorban, ami lehetővé teszi az egységes feldolgozást. A -1 érték jelzi, hogy ez nem valódi híd, hanem egy csúcs súlya.

Az *ans* tömb végső értéke a következőkből tevődik össze:

$ans[i]$ = feldolgozott hidak hatása + propagált érték

= az adott útvonalon a legnagyobb súlyú hídtól különböző hidak súlyainak összege

Ez pontosan megfelel a feladat követelményeinek.